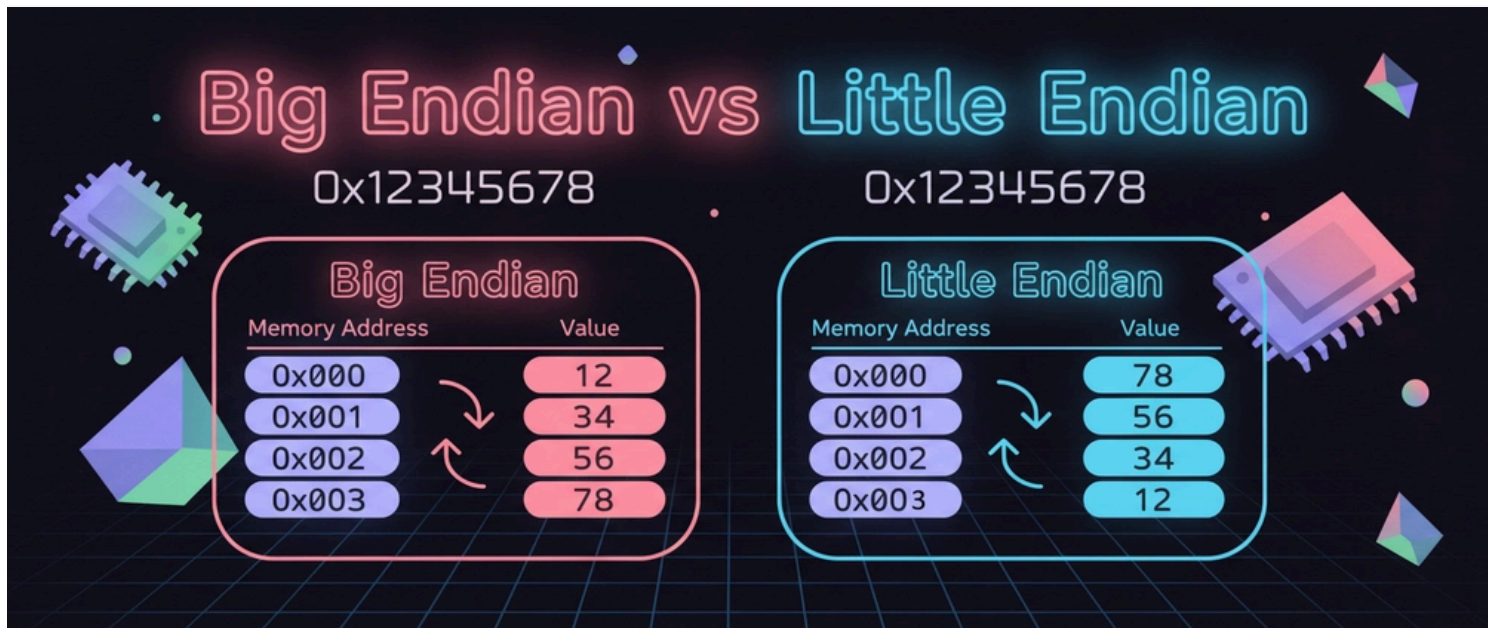


**Aman Prasad**

Posted on 12 فبراير



2

Understanding Endianness: Little-Endian vs Big-Endian

#discuss #beginners #learning #architecture

Endianness refers to the order in which bytes are arranged and stored in computer memory.

In simple terms, endianness decides **which byte is stored at the lowest memory address**: the most significant byte (MSB) or the least significant byte (LSB).

A simple analogy:

Think of endianness like reading direction.

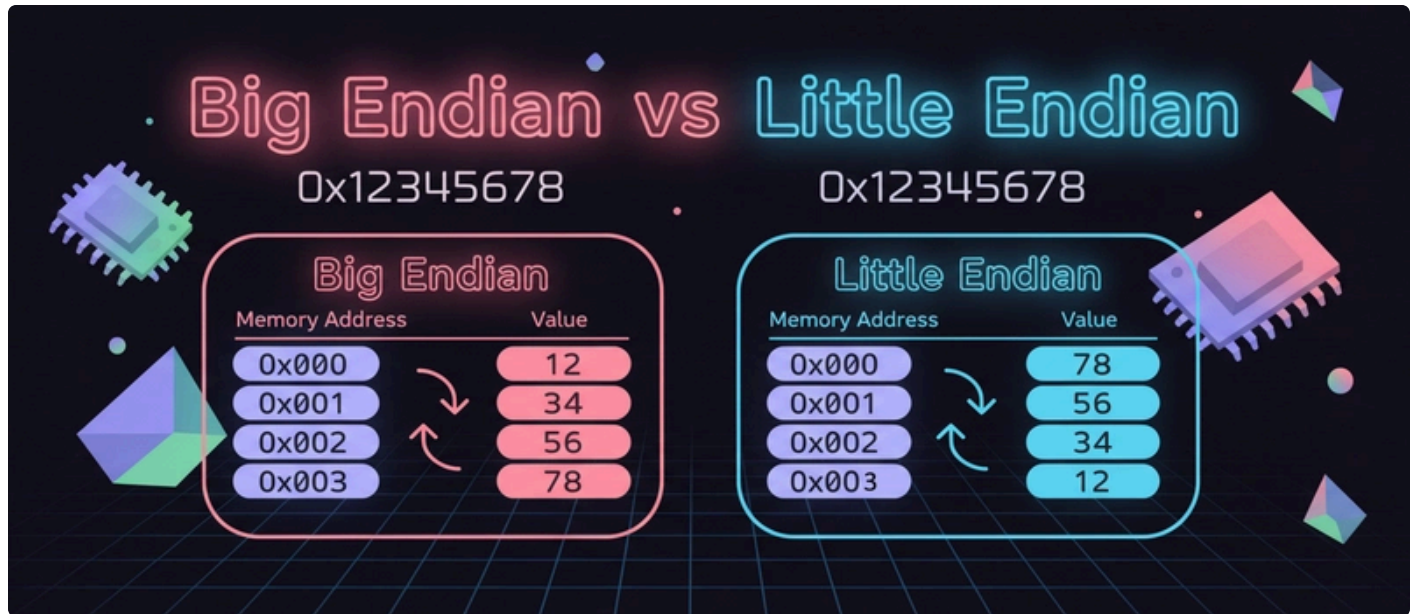
Some languages read **left to right**, while others read **right to left**. Both convey the same information but only if you know the rule beforehand.

Similarly, computers need a defined rule to interpret multi-byte values correctly.

The Two Main Types of Endianness

Most modern systems use one of two byte-ordering schemes.

To illustrate, consider storing the 4-byte hexadecimal value: `0x12345678`.



Big-Endian

In Big-Endian systems, the **most significant byte** (MSB) is stored at the lowest memory address.

This format is often considered more *human-readable* because it matches how we write numbers.

Memory Address	Stored Byte
Address +0	0x12 (MSB)
Address +1	0x34
Address +2	0x56
Address +3	0x78 (LSB)

It is used in Networking protocols, older mainframes and legacy systems.

Little-Endian (LE)

In Little-Endian systems, the **least significant byte** (LSB) is stored at the lowest memory address. This layout aligns well with how CPUs perform arithmetic internally.

Memory Address	Stored Byte
Address +0	0x78 (LSB)
Address +1	0x56
Address +2	0x34
Address +3	0x12 (MSB)

It is used by Intel (x86), AMD, and most modern desktops, laptops and embedded MCUs.

Bi-Endianness

Many modern processors, like **ARM**, are actually **Bi-endian**. This means they can be configured to operate in either Big-Endian or Little-Endian mode depending on the operating system's requirements.

In practice, most modern ARM systems run in **little-endian mode**.

Why Does It Matter?

In high-level languages like Python or Java, endianness is usually hidden.

However, it becomes critical in the following cases:

- **Networking:** The internet uses big-endian. Without proper byte conversion, data sent from a little-endian system will be misinterpreted.
- **Binary File Sharing:** Opening a binary file created on a big-endian system on a little-endian machine can corrupt values unless handled correctly.
- **Low-Level Programming:** In C, assembly, or embedded systems, incorrect assumptions about byte order lead to subtle and dangerous bugs.

Detecting Endianness Using C

```
#include <stdio.h>
#include <stdint.h>
```

```
int main() {  
    uint16_t x = 1;  
    if (*(uint8_t*)&x == 1){  
        printf("Little Endian");  
    }  
    else{  
        printf("Big Endian");  
    }  
    return 0;  
}
```

Step by step explanation

1. The Variable `uint16_t x = 1`

- `uint16_t` is a **16-bit (2-byte)** integer
- The numeric value is `1`, but memory must store it using **two bytes**

Possible memory layouts:

Endianness	Memory Bytes
Big-Endian	00 01
Little-Endian	01 00

2. The pointer cast `(uint8_t*)&x`

This line does three things:

1. `&x` → gets the memory address of `x`
2. `(uint8_t*)` → treats that address as a pointer to a **single byte**
3. Dereferencing reads the **first byte in memory**

3. if `*((uint8_t*)&x == 1)*`

4. **If the first byte is 1**: the least significant byte is stored first. The system is **Little-Endian**.

5. **If the first byte is 0**: the most significant byte is stored first. The system is **Big-Endian**.

A Subtle Advantage of Little-Endian

One often-mentioned but rarely explained advantage of little-endian systems is:

The same value can be read from memory at different widths using the same base address.

This works because, in little-endian memory, the **least significant byte is stored at the lowest address**.

Example

```
uint32_t x = 0x12345678;
```

Little-endian memory layout:

Memory Address	Stored Byte	Significance
$\&x + 0$	0x78	Least Significant Byte (LSB)
$\&x + 1$	0x56	
$\&x + 2$	0x34	
$\&x + 3$	0x12	Most Significant Byte (MSB)

$\&x$ always points to the **lowest memory address** ($\&x + 0$).

Now, reading from the same address:

Read size	Expression	Result
8-bit	$*(uint8_t*)&x$	0x78
16-bit	$*(uint16_t*)&x$	0x5678
32-bit	$*(uint32_t*)&x$	0x12345678

The starting address never changes only the read size does. Increasing the width naturally reveals more significant bytes.

Why Do We Care About the *Lower Bytes*?

This raises an important question:

If big-endian systems expose the upper part of a number first, why do CPUs and programmers care so much about the lower bytes?

The answer lies in **how arithmetic works**.

In any positional number system, the **least significant bits form the foundation of the value**, while higher bits only add scale.

For example: `0x12345678`

- The lower byte (`0x78`) controls changes of ± 1
- The upper byte (`0x12`) only affects large magnitude

All arithmetic operations addition, subtraction, multiplication **start from the least significant byte** and propagate upward using carry.

Addition and Subtraction

When adding multi-byte numbers, the CPU must process the **least significant byte first** to determine whether a carry occurs.

Example:

```
Carry:           1 1
Value A:   0x0 0 F F
+ Value B:   0x0 0 0 1
-----
Result:     0x0 1 0 0
```

1. LSB addition: `0xFF + 0x01 = 0x00` (carry = 1)
2. Next byte uses the carry to produce `0x01`

In little-endian systems, the LSB is fetched first, allowing computation to begin immediately while higher bytes are fetched in parallel. This was especially important on early 8-bit and 16-bit processors and strongly influenced CPU and compiler design.

Why Big-Endian Still Exists

If little-endian fits computation so well, why does big-endian persist?

The reason is **legacy and standardization**, not performance.

Big-endian is used in:

- Networking (TCP/IP)
- File formats like **JPEG** and **PNG**
- Older architectures and mainframes

Once a format or protocol is defined, changing its byte order would break compatibility with existing data and software.

Modern Reality

On modern systems:

- CPU arithmetic happens in registers (no endianness)
- Caches and pipelines hide memory order
- High-level languages abstract it away

Today, **endianness is mostly a data-format concern**, not a CPU performance concern except in networking, embedded systems, and low-level code.

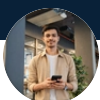
Reference

This discussion is inspired by community explanations on Stack Overflow:

<https://stackoverflow.com/questions/13926760/the-reason-behind-endianness>

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



Aman Prasad

ECE student focused on embedded systems, ARM Cortex-M, and low-level C/C++. Writing to understand how hardware and firmware really work.

LOCATION

India

EDUCATION

B.Tech in Electronics and Communication Engineering

JOINED

19 2025 دجنبر

More from [Aman Prasad](#)

The Inline Myth: Why the inline Keyword is Just a Suggestion

[#discuss](#) [#learning](#) [#c](#) [#beginners](#)

char str1[] = "hello world"; vs char *str2 = "hello world"; – The Memory Story Every C Programmer Must Know

[#discuss](#) [#c](#) [#learning](#) [#beginners](#)

MQTT Explained in Simple Terms: The Lightweight Protocol That Powers the Entire IoT World

[#discuss](#) [#beginners](#) [#learning](#) [#iot](#)
